LAB 1: Basics of ROS

By: Vy Tang

ID: 77659006

Objective

The objective of this report is to get exposure to Robot Operating System (ROS) by creating a node that interacts with TurtleSim. This lab focuses on creating a workspace, building a package, and implementing Python node that enables the program to take user inputs and control the turtle's movements shown below.

Robot Trajectories

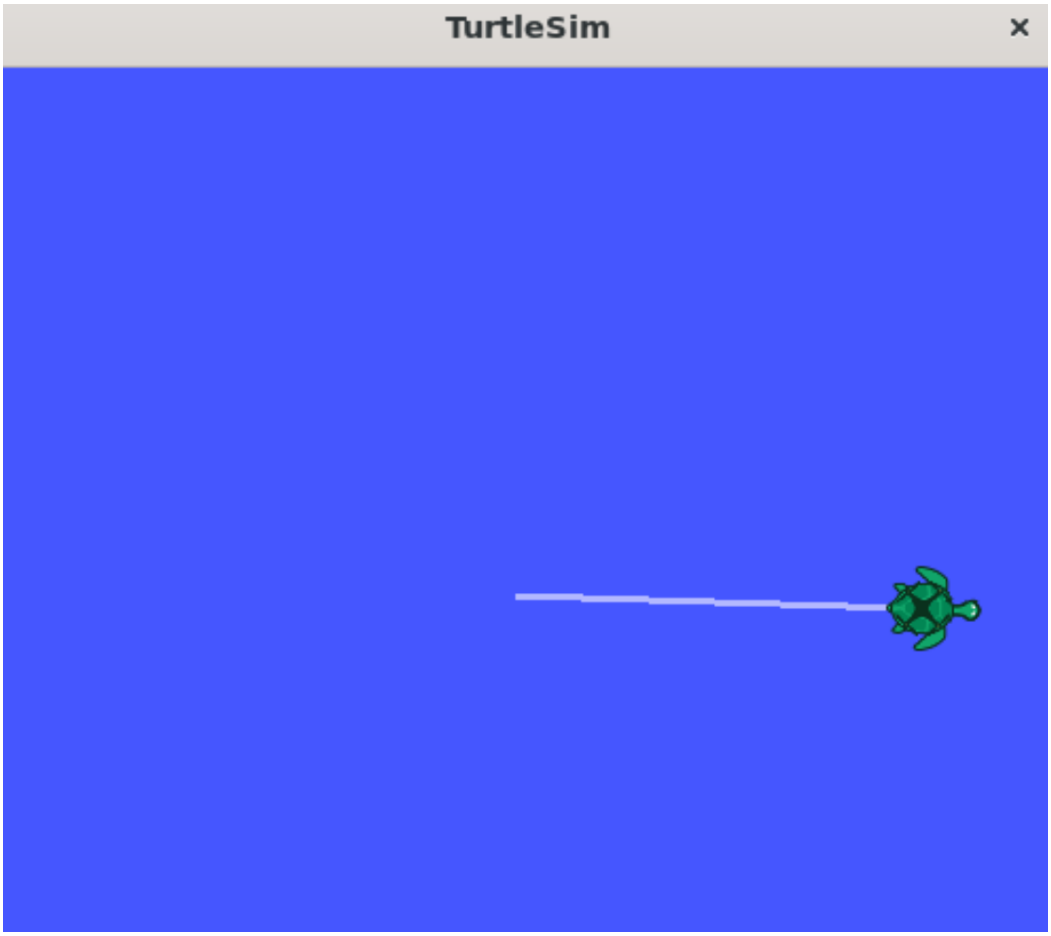

Fig 1. Robot's first trajectory

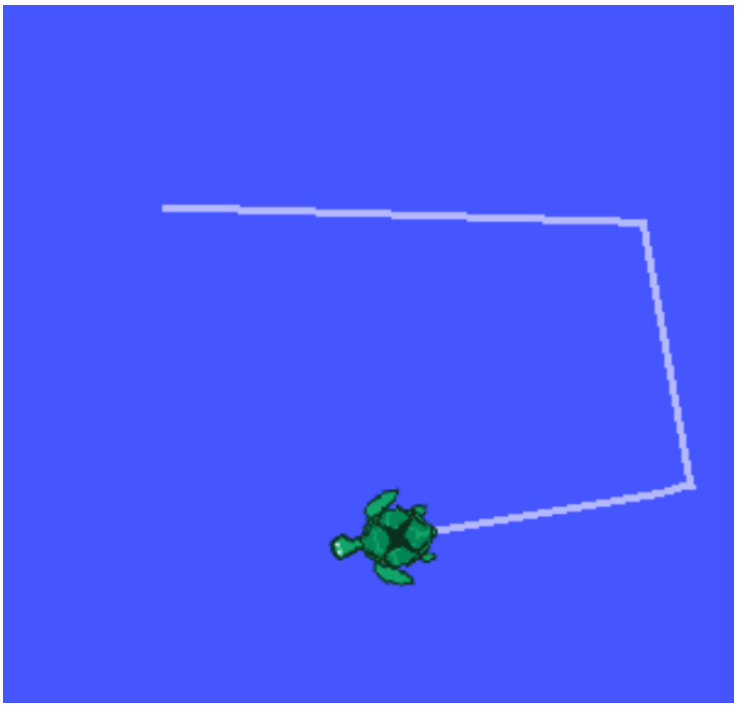

Fig 2. Robot’s second trajectory

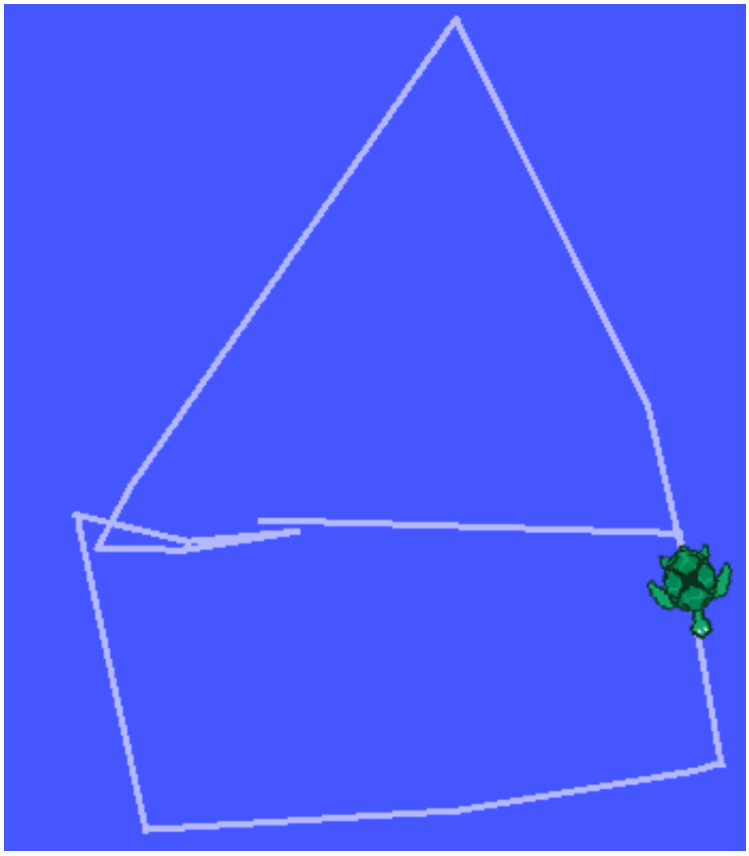

Fig 3. Robot’s final trajectory

Code Breakdown

```python
#!/usr/bin/env python3

import rospy
from geometry_msgs.msg import Twist
import curses

class TurtleTeleop:
    def __init__(self):
        # Initialize the ROS node
        rospy.init_node('my_teleop_node')
        self.pub = rospy.Publisher('/turtle1/cmd_vel', Twist, queue_size=10)
        self.twist = Twist()

        # Movement parameters
        self.linear_speed = 0.5  # m/s
        self.angular_speed = 0.5  # rad/s

    def send_command(self, linear, angular):
        """Publish movement command"""
        self.twist.linear.x = linear
        self.twist.angular.z = angular
        self.pub.publish(self.twist)
```

Select Indentation

```python

    def run(self):
        # Initialize curses for keyboard input
        stdscr = curses.initscr()
        curses.cbreak()
        stdscr.keypad(True)
        stdscr.nodelay(True)

        try:
            while not rospy.is_shutdown():
                key = stdscr.getch()

                # Handle key presses
                if key == ord('w'):  # Move forward
                    self.send_command(self.linear_speed, 0.0)
                elif key == ord('s'):  # Move backward
                    self.send_command(-self.linear_speed, 0.0)
                elif key == ord('a'):  # Rotate counterclockwise
                    self.send_command(0.0, self.angular_speed)
                elif key == ord('d'):  # Rotate clockwise
                    self.send_command(0.0, -self.angular_speed)
                elif key == ord('q'):
```

```
45                      # Quit the program
46                      self.send_command(0.0, 0.0)
47                      break
48
49
50          finally:
51              # Clean up curses
52              curses.nocbreak()
53              stdscr.keypad(False)
54              curses.echo()
55              curses.endwin()
56
57  if __name__ == '__main__':
58      try:
59          teleop = TurtleTeleop()
60          teleop.run()
61      except rospy.ROSInterruptException:
62          pass
```

In this lab I used Object-Oriented Programming. First, I created a class and initialized the nodes using the init.node and .publisher to initialize my nodes and declare the publisher from the rospy library. Next, I call the Twist command to create an empty Twist() to store the velocity commands in which I declared the next two lines where the angular speed and the linear speed is arbitrarily given as 0.5.

Then, I would define a helper function send_command to send the velocity commands. Inside the function, I would set the linear velocity previously defined and then publish those velocity command to the turtle.

The run() function, will use the curses library to initialize the curses library to be able to receive user input from the keyboard. The cbreak method essentially allows for instant key press for the user without them having to press enter every time they want to make an input.

The next step is to implement a while loop until the ROS program shuts down. Meanwhile it is running, the system will read the pressed key and checks for the conditions whether if the user presses either ‘W’ which corresponds to moving forward, ‘S’ which corresponds to moving backwards, ‘ A’ for rotating counterclockwise, and finally ‘D’ for rotating clockwise. Depending on which of these keys are pressed, it will either send a forward/backward command or a rotating command. Finally, if the user hits ‘Q’ the program will automatically exit.

The finally block ensures cleanup runs if error occurs and in this block, the nocbreak method restore normal terminal input buffering and then closes the curses window using the endwin method.

The final code block with an if-statement is for when the program ends to stop the curses function. It will also check if there is any launch failures .